

Assistente conversacional

Objetivo e base didática

O CardioIA simula a coleta inicial de informações fictícias em saúde. A Parte 1 usa IBM Watson Assistant com intenções, entidades, nós de diálogo e contexto; a Parte 2 usa Flask e HTML. A referência é PCV, capítulo 10, pp. 17-37 e 46-49. Docker foi acrescentado como requisito do projeto.

Caminho da mensagem

Navegador > Flask > sessão da API v2 do Watson > reconhecimento de intenção e entidades > nó de diálogo > resposta e contexto > resumo na interface. O backend conserva o identificador remoto em SQLite e usa um token opaco na sessão do navegador. Cada perfil possui contexto separado; abas do mesmo perfil compartilham a sessão.

Fluxos implementados

O catálogo contém 14 intenções e 70 exemplos de treinamento: saudação, ajuda, sintomas, medição, histórico, termos educativos, limites de conduta, urgência declarada, resumo, confirmação, correção, reinício, encerramento e extração. Cinco entidades reconhecem sintomas, tipo de medição, campo de correção, pressão com mmHg e frequência com bpm. As entidades de sintomas não são tratadas automaticamente como sintomas presentes.

Exemplo de interação

Usuário: Quero registrar minha pressão. Watson: Informe a pressão completa em mmHg, por exemplo 120/80 mmHg. Usuário: 120/80 mmHg. Watson registra o valor literal e pede conferência. Ao solicitar o resumo, a interface exibe os valores informados. Uma confirmação explícita atualiza o indicador; uma correção retoma a coleta. Relatos livres recebem uma pergunta sobre início/duração.

Contexto e exceções

As variáveis guardam etapa, relato, duração, histórico, medições e confirmação. Comandos globais precedem a captura de texto livre. Formatos incompletos recebem orientação; mensagens desconhecidas têm fallback e opções após repetição. Reiniciar cria nova sessão e remove o estado anterior. Expiração remota retorna erro controlado e permite iniciar outra conversa. O encerramento do diálogo termina o fluxo, e o botão Nova conversa cria um contexto limpo.

Integração e apresentação

O Flask serve a página e as rotas de conversa, extração e monitoramento. Valida tipo, tamanho e conteúdo das requisições; usa proteção CSRF e renderização por textContent. O resumo mostra somente campos permitidos. Ausências permanecem vazias; uma negação em um relato não vira diagnóstico. Segredos vêm do ambiente, nunca de JavaScript ou do JSON da skill.

Execução reproduzível

`docker compose --profile automacao up --build -d` inicia aplicação, MongoDB e robô. A aplicação fica em `localhost:5000`. A imagem usa Python 3.12 e Gunicorn sem debug, executados como usuário sem privilégios. Os volumes conservam contexto, contagem de chamadas, medições e registros. O endpoint `/health` verifica o processo HTTP e não testa credenciais externas.

Validação e estado da entrega

Foram aprovados 26 testes de backend/extração e cinco do robô em containers. O fluxo HTTP com APIs reais validou medições, confirmação, correção, isolamento entre clientes, Gemini integrado ao Watson e reinício. O Watson classificou 38 de 42 frases inéditas conforme a referência (90,5%); as quatro diferenças estão documentadas. A skill oficial foi exportada pela API v2 com 14 intenções, cinco entidades e 34 nós. O SDK usa `assistant_id`, `environment_id` e `user_id` opaco. Evidências e limitações constam em `docs/VALIDACAO.md`. A publicação no GitHub ficará a cargo do usuário.

Limites

Trata-se de protótipo educacional com dados sintéticos. Não fornece diagnóstico, prescrição ou avaliação de emergência. A extração Gemini e a análise estatística são extensões separadas; a responsabilidade pela condução da conversa permanece no Watson. A interface solicita revisão dos dados antes de confirmação.

Referências e arquivos

PCV: 2TIAOA - Fase 5 - Cap10 - Arquitetura Cognitiva dos LLMs Modernos, pp. 17-37, 46-49 e 60-63.
Implementação: `app.py`, `services/watson_service.py`, `services/clinical_summary.py`, `watson/build_skill.py` e `templates/index.html`. Instruções completas e variáveis: `README.md` e `.env.example`.